

Event Booking & Ticket Validation API

Objective

You need to build a backend that simulates an event booking platform. The system allows users to:

- Register / log in
- View and book events
- Validate tickets

Use in-memory arrays for all data (no DB). Authentication is handled via JWT tokens.

Instructions

You must:

- Store all data in memory (let users = [], let events = [], etc.).
- Passwords can be stored in plain text (no hashing needed for this test).

Routes to Implement

1. POST /auth/register

Registers a new user.

Request:

```
{
  "name": "ved", "email": "ved@gmail.com", "password": "123", "role": "organizer"
}
```

Response:

```
{
  "message": "User registered successfully",
  "userId": 1
}
```

2. POST /auth/login

Request:

```
{
  "email": "ved@example.com",
  "password": "1234"
}
```

Response:

3. POST /events

Request Body:

Response:

4. GET /events

Response:

5. POST /events/:id/book

Example Request:

Response:

```
{
  "message": "Ticket booked successfully",
}
```

```
"ticketCode": "TCK-98217"
}
```

6. GET /my-bookings

Returns all tickets booked by the logged-in user.

Response:

```
[
  {
    "event": "Tech Tournament",
    "ticketCode": "TCK-98217",
    "validated": false
  }
]
```

7. POST /tickets/validate

Validate a ticket at the event entrance.

Request:

```
{
  "ticketCode": "TCK-98217"
}
```

Response (if valid):

```
{
  "message": "Ticket validated successfully",
  "event": "Tech Tournament",
  "userId": 1
}
```

Response (if invalid or already validated):

```
{
  "error": "Invalid or already validated ticket"
}
```

Note: Organizer can only validate tickets for their own events.

8. GET /events/:id/attendees

List all validated tickets for your event.

Response:

```
{
  "event": "Solana Mixer",
  "attendees": [
    { "userId": 1, "ticketCode": "TCK-98217" }
  ]
}
```